



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

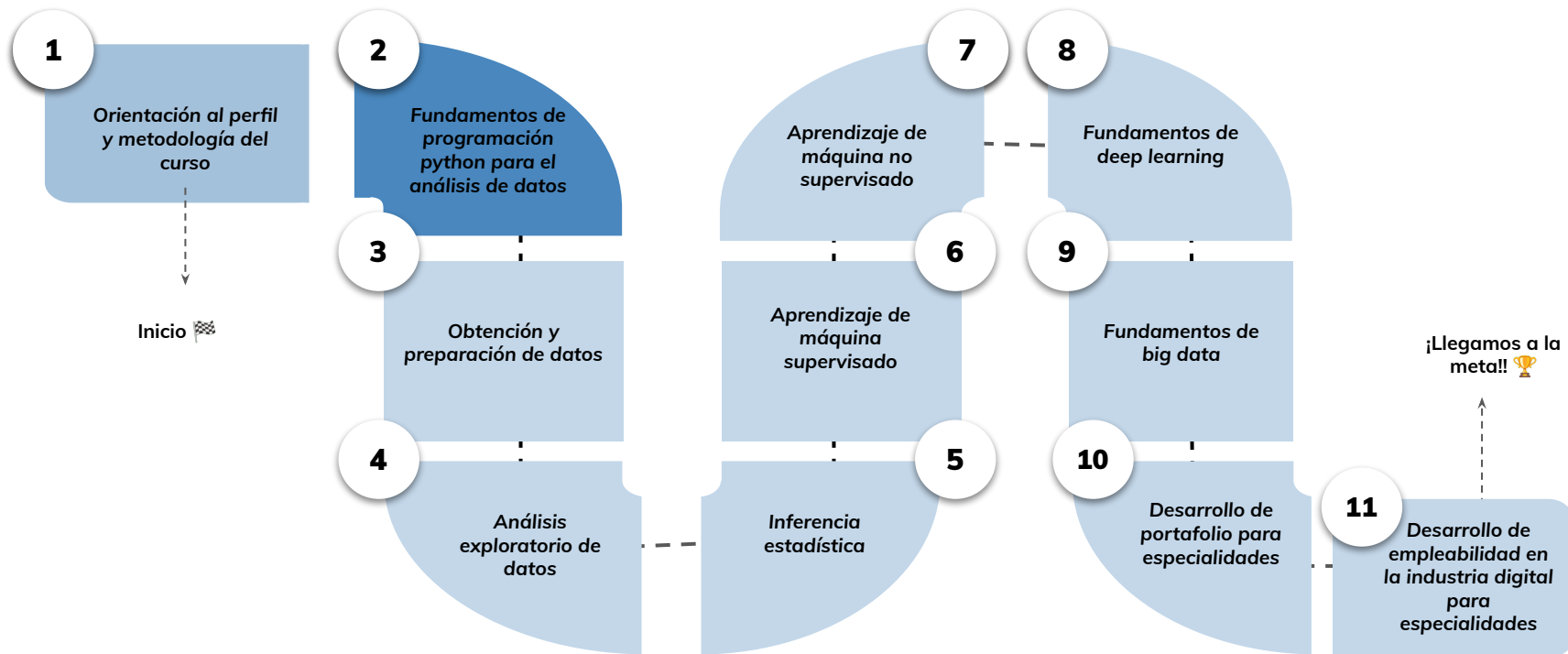


Programación orientada a objetos en python- Parte 1

Aprendizaje Esperado 7: Codificar una rutina utilizando clases provistas para la resolución de un problema simple de acuerdo al paradigma de orientación a objetos en el entorno python.

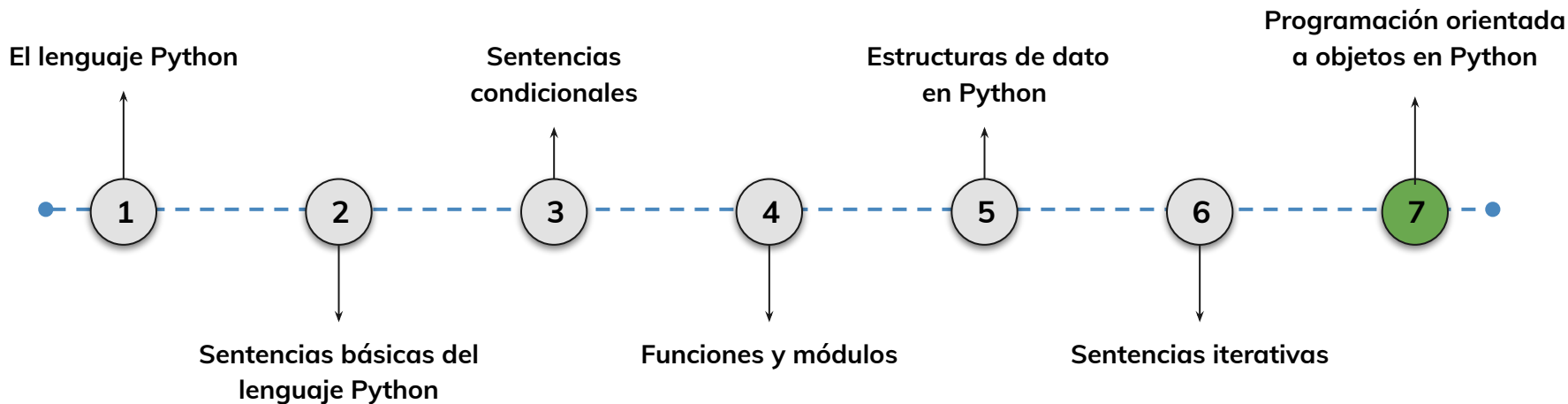
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

7.

Aplicación del paradigma
orientado a objetos en
Python.

Trabajaremos en los fundamentos de la Programación Orientada a Objetos (POO). Aprenderemos a crear clases con atributos y métodos, instanciar objetos, trabajar con el constructor `__init__()` y utilizar principios clave como la abstracción y la encapsulación para organizar código de forma modular y reutilizable.

Introducción al paradigma
de orientación a objetos

conceptos y beneficios

Clases, objetos, atributos y
métodos

Instanciación de objetos

uso de métodos

Comparación con
programación estructurada

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es una clase y cómo se relaciona con un objeto
- Definir atributos y métodos en una clase usando self
- Instanciar objetos con valores personalizados
- Aplicar principios de encapsulación y abstracción
- Resolver un problema simple modelando datos y comportamientos con clases



Repaso clase anterior

¿Quedó alguna duda?



En la clase anterior trabajamos :

- Aplicamos sentencias iterativas como `for`, `while`, `enumerate()` y `zip()`
- Recorrimos estructuras complejas para analizar y transformar información
- Usamos comprensiones para generar listas y diccionarios de manera eficiente
- Creamos un análisis académico automatizado con patrones de repetición

Programación orientada a objetos en python



Programación orientada a objetos en python

La programación orientada a objetos (OOP, por sus siglas en inglés) es uno de los paradigmas de programación más utilizados en la actualidad. Este enfoque permite estructurar el código en torno a objetos, que son representaciones de entidades del mundo real con características y comportamientos específicos.



Fundamentos de programación Python para el análisis de datos

A través de la OOP, los programadores pueden modelar sistemas complejos de forma más intuitiva y organizada, mejorando la modularidad, el mantenimiento y la reutilización del código.

Python, aunque es un lenguaje multiparadigma, ofrece un soporte sólido para la programación orientada a objetos, permitiendo que los desarrolladores trabajen con clases y objetos, encapsulen datos y comportamientos, y apliquen principios fundamentales como la abstracción y el encapsulamiento.



Data Analysis Vis

en by to latent éats Aertesis and Python: Tatons Veamcl

1 Stad p Website Traffic

- Suamiment astaic tranecâtem er thine frest sunpried ceviel ililit, engring, ant arter tybroet doorpiaten to dotespensing or uitore datar acuritedatarication is Python s afforue date website frenall exproctated to sartor isures rnstramegly.

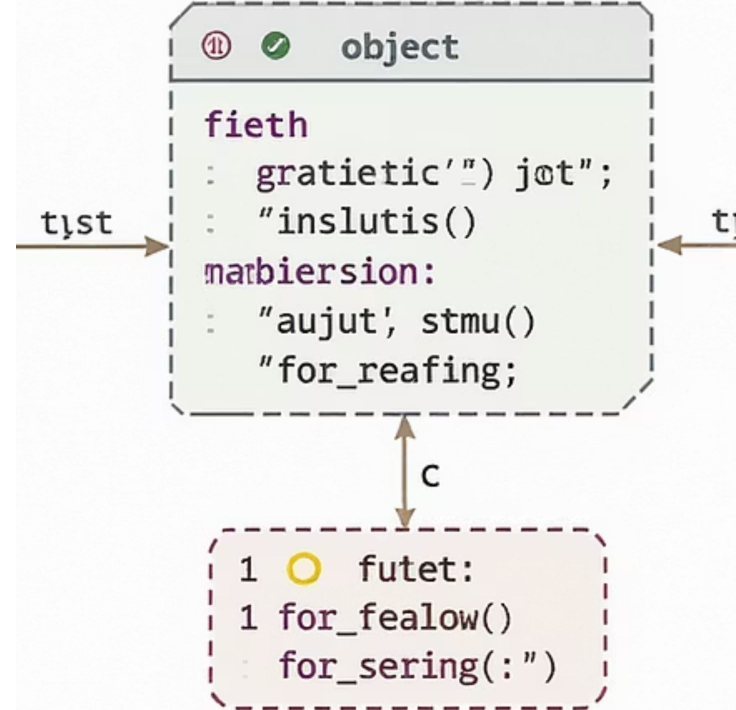




En Qué Consiste el Paradigma de Orientación a Objetos

La **orientación a objetos** es un paradigma de programación que organiza el código en torno a "objetos" que representan entidades del mundo real o conceptos abstractos. Cada objeto tiene atributos que definen su estado y métodos que determinan su comportamiento.

Oriented programming Paradigm





Enfoque en objetos

A diferencia de la programación estructurada o funcional, donde el enfoque está en las funciones o procedimientos, la OOP permite agrupar datos y funciones en unidades cohesivas llamadas clases.

Modularidad

Los programas basados en objetos tienden a ser más fáciles de mantener, extender y reutilizar, ya que los objetos pueden interactuar entre sí sin afectar otras partes del programa.

Colaboración

Esta modularidad es clave en proyectos grandes, donde diferentes equipos pueden trabajar en distintos objetos sin interferir en el trabajo de los demás.





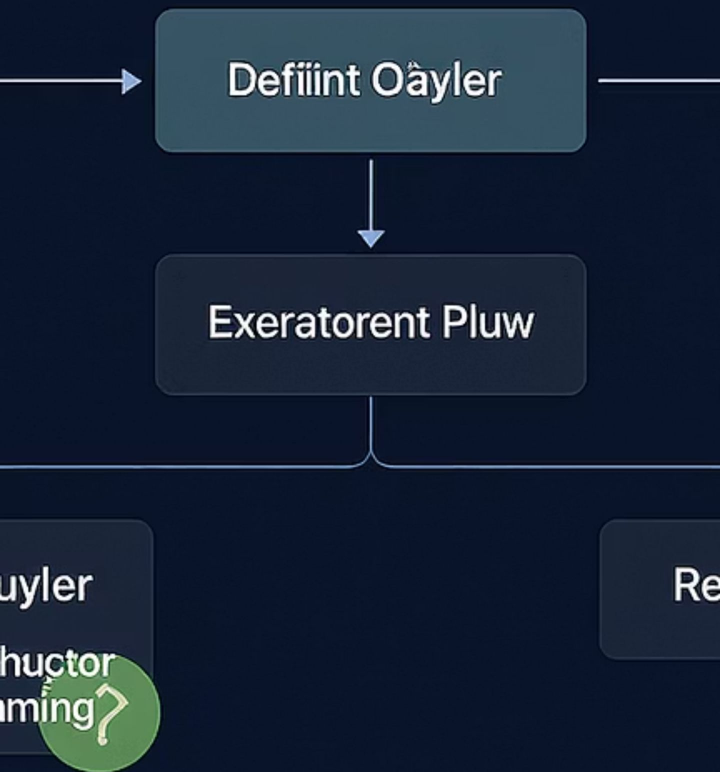
Python como lenguaje multiparadigma

Python es un lenguaje multiparadigma, pero ofrece un soporte nativo robusto para la OOP. Esto facilita la creación y manipulación de objetos a través de una sintaxis accesible y herramientas avanzadas como herencia, polimorfismo y encapsulamiento.

A través de la OOP, Python permite una mayor organización y claridad en el código, especialmente en proyectos grandes y complejos.



Object-Oriented Programming



Principios de diseño en OOP

El paradigma de OOP fomenta el uso de principios de diseño que permiten que el código sea adaptable y fácil de entender. Entre estos principios están la encapsulación, la abstracción, la herencia y el polimorfismo.

Estos principios ayudan a dividir los problemas en componentes más manejables y reutilizables, promoviendo así el desarrollo de aplicaciones más eficientes y menos propensas a errores.

```
on  
ecians(  
at >(: "class crediction"  
L.({});  
ass: "object of corject.fal  
or (f( 1');  
.or({'});
```

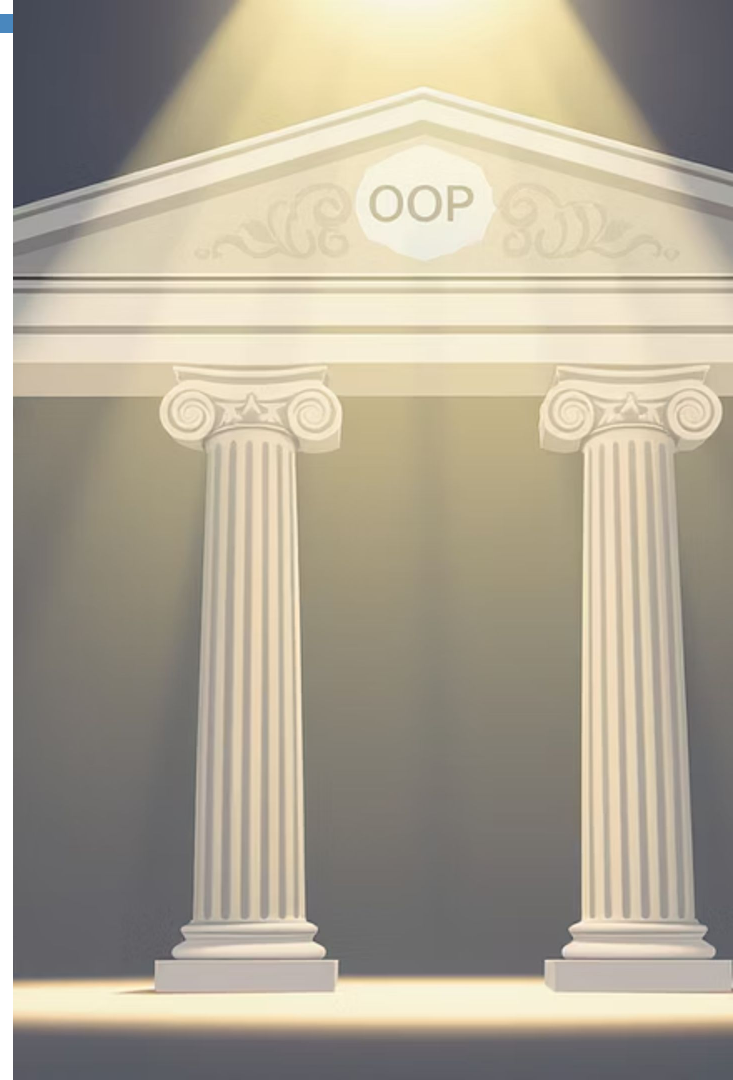
OOP en Python: Un enfoque natural

En Python, la OOP es un enfoque natural y poderoso que mejora la claridad y escalabilidad del código. Comprender cómo aplicar el paradigma de orientación a objetos en Python es fundamental para cualquier programador que desee crear aplicaciones complejas y mantener una arquitectura de código sólida y extensible.



Principios Básicos de la Programación Orientada a Objetos

La programación orientada a objetos se basa en cuatro principios fundamentales: **encapsulación**, **abstracción**, **herencia** y **polimorfismo**. Estos principios son los pilares que guían la construcción de sistemas orientados a objetos y determinan cómo los objetos interactúan y se relacionan dentro del programa.

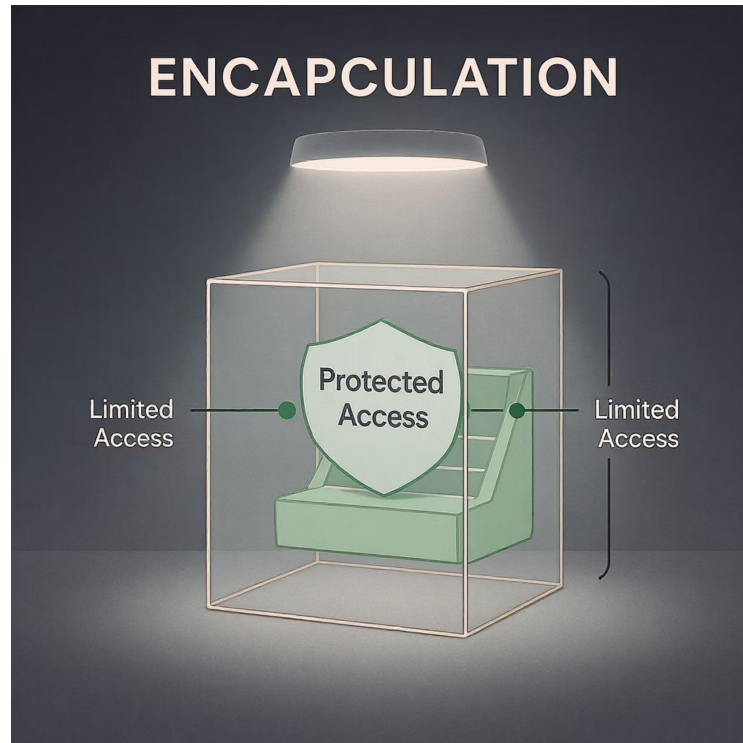


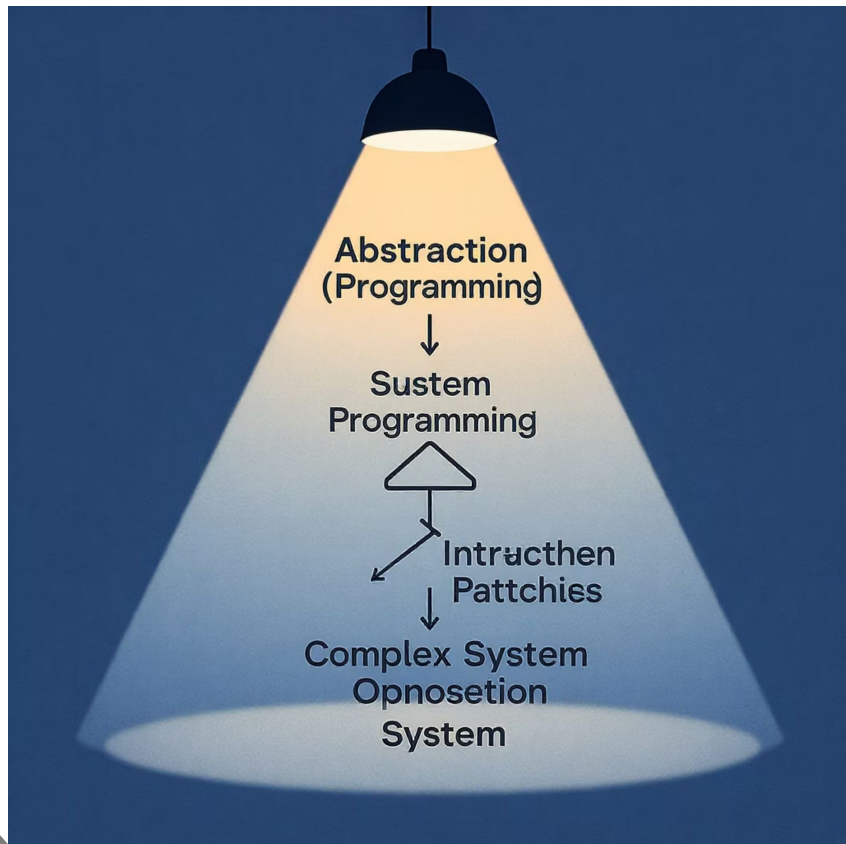


Encapsulación

La **encapsulación** implica ocultar los detalles internos de un objeto y permitir el acceso solo a través de métodos definidos. Esto ayuda a proteger el estado interno de un objeto y evita que sea modificado de manera incorrecta.

En Python, la encapsulación se puede implementar mediante la creación de métodos `get` y `set` o utilizando convenciones como los guiones bajos para señalar atributos privados.





Abstracción

La **abstracción** se refiere a la capacidad de representar características esenciales de un objeto sin exponer detalles complejos o innecesarios.

En Python, la abstracción se logra mediante la creación de clases que representan ideas o entidades sin necesidad de revelar su implementación interna.

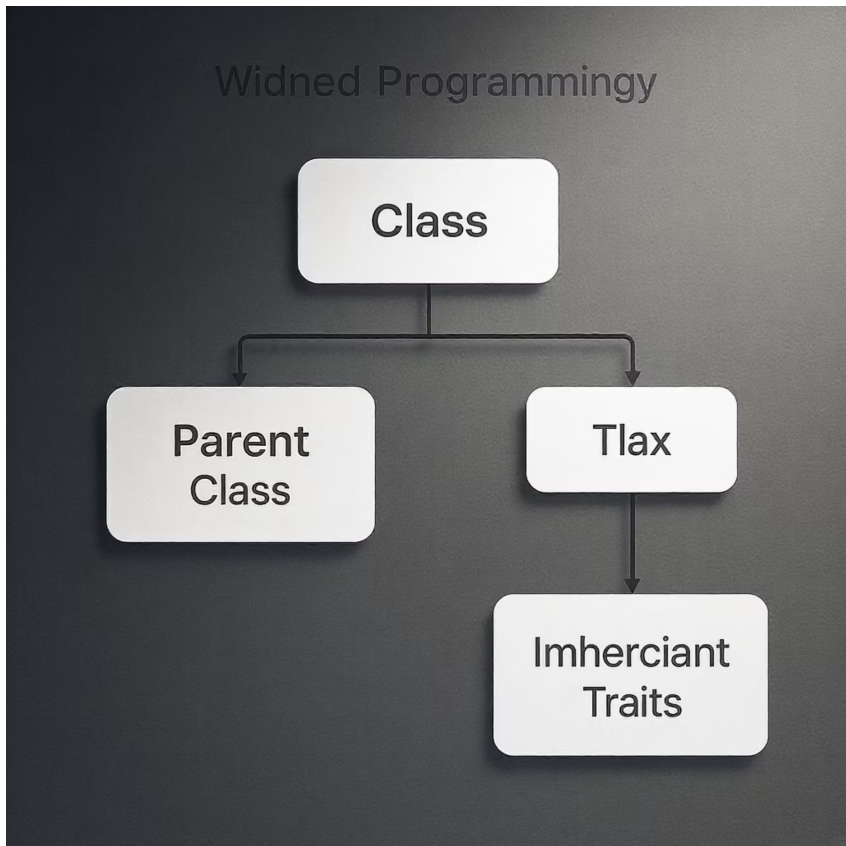
Por ejemplo, al interactuar con un automóvil, un usuario solo necesita saber cómo arrancarlo y manejarlo, sin preocuparse por los detalles de su motor.



Herencia

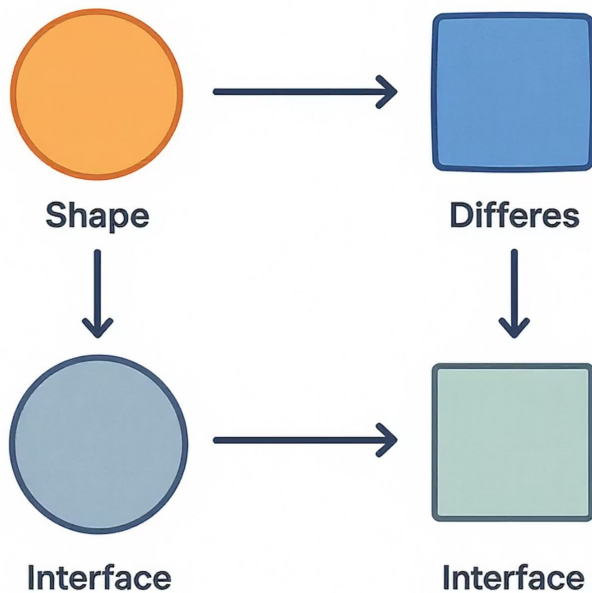
La **herencia** permite que una clase derive características de otra clase. Esto promueve la reutilización de código y facilita la creación de relaciones jerárquicas entre objetos.

En Python, una clase puede heredar atributos y métodos de otra, permitiendo que los desarrolladores creen clases especializadas basadas en clases existentes.





Polymorphism



Polimorfismo

El **polimorfismo** es la capacidad de los objetos de responder a la misma interfaz de manera diferente, dependiendo de su tipo específico.

En términos simples, permite que métodos con el mismo nombre funcionen de manera diferente en distintas clases.

En Python, el polimorfismo se utiliza para crear métodos con nombres similares en diferentes clases que cumplen funciones diversas.

```
classes_fomotion{  
emp(IP1 classes  
_initition-in_classes(;  
er cluses {  
{quce>}f intfIyation= initce(ritto  
{tate"ingened"tur life);  
rfaut_cwate; inthe_souce,_steacs));  
rtla(_ing_com());  
thar(leass_inlley;  
one ftuect_classer_itor);
```

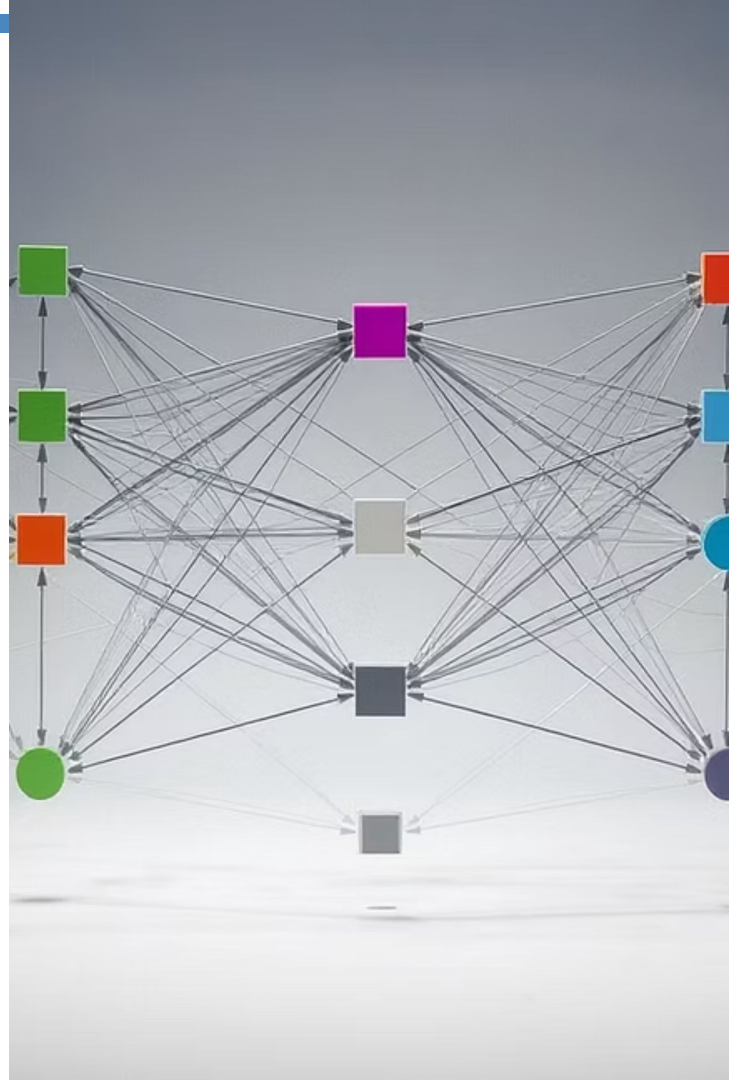
Aplicación de principios OOP en Python

Comprender y aplicar estos principios en Python facilita la creación de aplicaciones más organizadas y escalables. Estos principios son el núcleo de la OOP y constituyen la base para el diseño de software moderno.



Abstracción y Ocultamiento

La **abstracción** es el proceso de simplificar la complejidad del mundo real mediante la creación de modelos que capturen solo los aspectos esenciales de un objeto. En programación, la abstracción permite definir objetos que representan ideas generales, como "automóvil" o "empleado", sin detallar sus componentes internos específicos.





Beneficios de la abstracción



Simplificación

La abstracción permite trabajar con objetos sin conocer sus detalles internos, facilitando un diseño limpio y enfocado en las funcionalidades esenciales.



Enfoque en lo importante

Al definir una clase, el programador decide qué atributos y métodos son necesarios para representar la funcionalidad deseada, sin necesidad de exponer cómo funcionan esos métodos internamente.



Interfaz clara

Por ejemplo, una clase Coche podría tener métodos como arrancar o detenerse que ocultan la lógica interna de cómo funcionan, y el usuario solo necesita invocarlos sin preocuparse por los detalles.





Ocultamiento en Python

El **ocultamiento**, o encapsulación, es el mecanismo que permite proteger los datos internos de un objeto de accesos o modificaciones externas no controladas.

Python implementa el ocultamiento utilizando convenciones, como el uso de un guion bajo al inicio de un nombre de atributo (`_atributo`) para indicar que es privado y no debe accederse directamente desde fuera de la clase.

```
private cytlen(latl('')
attriztiw= {
}
napgrdaton attitutie({
mape.entcrizled
}
```





Convenciones de privacidad en Python

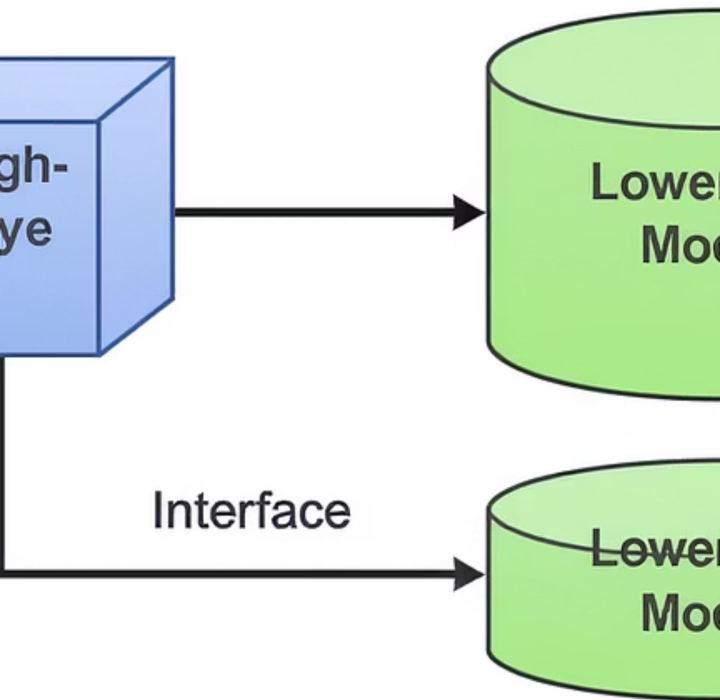
Aunque Python no impone una privacidad estricta como otros lenguajes, esta convención ayuda a señalar cuáles elementos deben considerarse privados.

La abstracción y el ocultamiento trabajan juntos para lograr que el código sea más seguro y menos propenso a errores, permitiendo que los usuarios de una clase solo interactúen con los métodos y atributos públicos y de alto nivel.



```
tribute {" {  
tributé") ="name");  
  
rot =syttion);  
ntibl(#=respersentvering.)  
or-mlation(;;  
vicatribation:  
: privats corver-same)
```

BENEFITS OF ABSTRACTION



Beneficios de la abstracción y ocultamiento

Al ocultar la complejidad innecesaria, estos principios promueven la claridad en el diseño y la simplicidad en el uso de los objetos.

En conclusión, la abstracción y el ocultamiento son prácticas esenciales en la OOP, ya que permiten crear interfaces limpias y centrarse en lo que un objeto hace en lugar de cómo lo hace. Esto facilita la escalabilidad y la reutilización del código en Python, permitiendo que los programas sean más robustos y fáciles de mantener.



Clases y Objetos

En la OOP, una **clase** es una plantilla o modelo que define las propiedades y comportamientos comunes a todos los objetos de un mismo tipo. Es una especie de plano que describe cómo se debe construir un objeto, especificando atributos y métodos que estos objetos tendrán.

Por otro lado, un **objeto** es una instancia concreta de una clase, es decir, un elemento creado a partir de la plantilla de la clase con sus propios valores y características.





Definición de clases en Python

En Python, las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase y dos puntos.

Dentro de una clase, se pueden definir atributos y métodos que determinarán el estado y comportamiento de los objetos creados a partir de esa clase.

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
    def ladrar(self):
        print("¡Guau!")

mi_perro = Perro("Rex", 3)
mi_perro.ladrar()
```



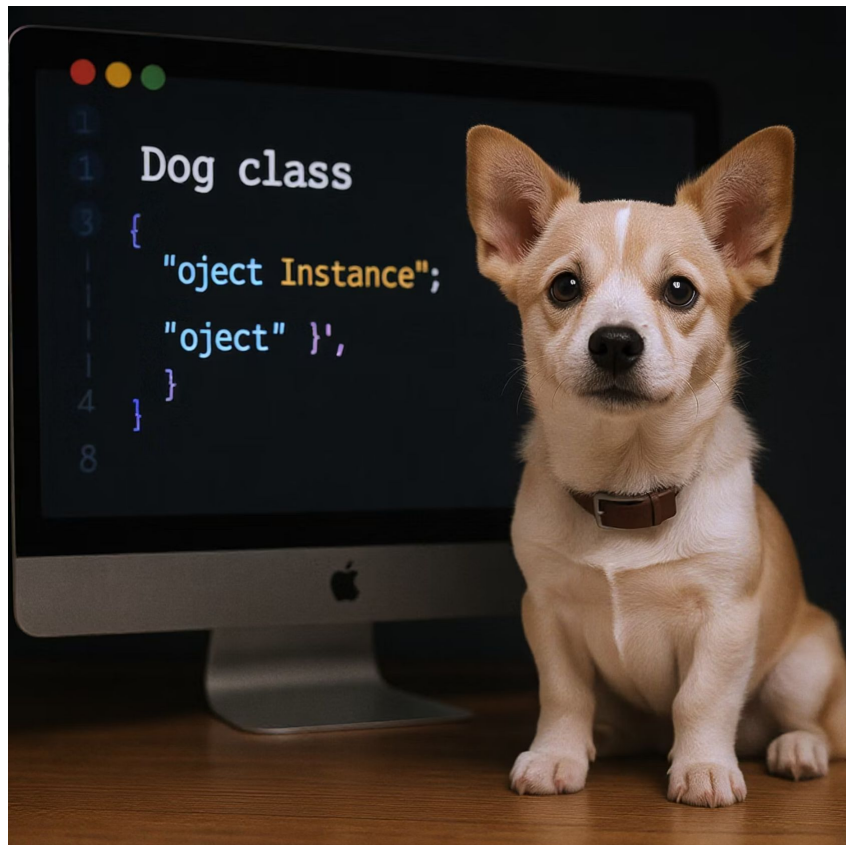


Clases y objetos:

Ejemplo explicado

En el ejemplo anterior, **Perro** es la clase y **mi_perro** es un objeto (o instancia) de esa clase.

Al crear una instancia de Perro, se asignan valores específicos a sus atributos nombre y edad. Luego, se puede llamar al método ladrar() del objeto mi_perro, que produce un comportamiento específico (en este caso, imprimir "¡Guau!").





Beneficios de las clases



Organización

Las clases permiten organizar el código en unidades reutilizables y coherentes.



Estado independiente

Cada objeto creado a partir de una clase puede tener su propio estado, definido por los valores de sus atributos, lo que facilita el manejo de datos y comportamientos en aplicaciones complejas.



Modularidad

La creación de clases facilita la abstracción y encapsulación, permitiendo que el código sea modular y fácil de entender.





Interacción entre objetos

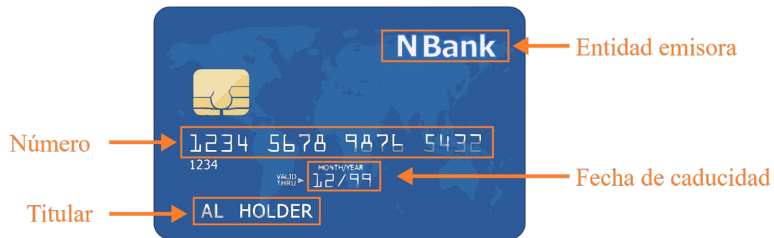
Los objetos en Python pueden interactuar entre sí, lo cual es fundamental en la OOP. Cada objeto mantiene su propio estado y puede ejecutar sus propios métodos, lo que permite modelar relaciones entre objetos y crear sistemas de objetos que se comunican y colaboran para realizar tareas complejas.





Estado y Atributos

Atributos



Métodos

Activar



Pagar



Anular



El **estado** de un objeto en la OOP se refiere a los valores actuales de sus atributos, que definen sus características en un momento dado. Los **atributos** son las propiedades que describen un objeto y permiten que cada instancia de una clase tenga su propio estado único.

En Python, los atributos se definen en el constructor de la clase, que normalmente es el método `__init__`, y se accede a ellos utilizando la palabra clave `self`.

Fuente: [Programación Orientada a Objetos](#)

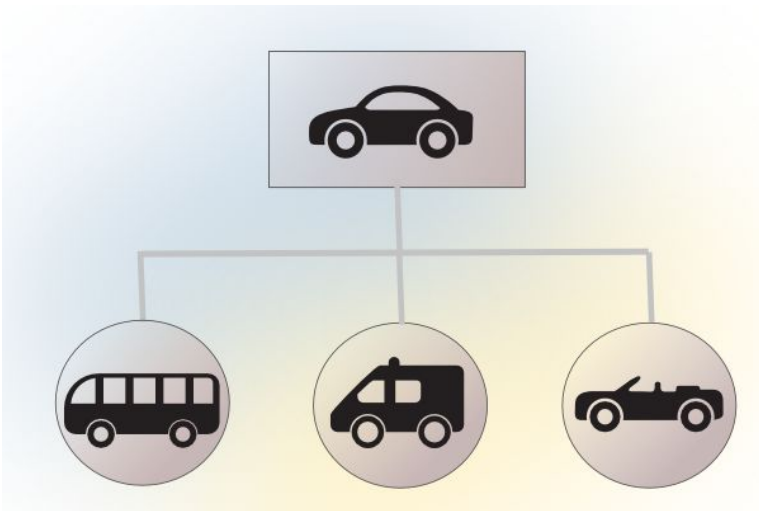


Ejemplo de estado y atributos

Por ejemplo, en la clase Perro, los atributos nombre y edad definen el estado de un objeto Perro. Si creamos dos objetos Perro con nombres y edades diferentes, cada uno tendrá un estado único y separado:

```
perro1 = Perro("Rex", 3)  
perro2 = Perro("Bella", 5)
```





Objetos con estados independientes

En este caso, perro1 y perro2 son objetos diferentes, cada uno con su propio estado, a pesar de haber sido creados a partir de la misma clase.

Los atributos permiten personalizar los objetos y representan las propiedades únicas de cada uno, lo cual es esencial para modelar elementos del mundo real.





Tipos de atributos en Python

Atributos públicos

Se pueden acceder y modificar directamente desde fuera de la clase.

Atributos privados

Se designan con un guion bajo (`_atributo`) para indicar que no deben ser modificados externamente. Esto es parte del concepto de encapsulación.

Atributos de clase

Son compartidos por todas las instancias de esa clase. Estos atributos se definen fuera del método `__init__` y pueden usarse para definir propiedades comunes a todos los objetos.



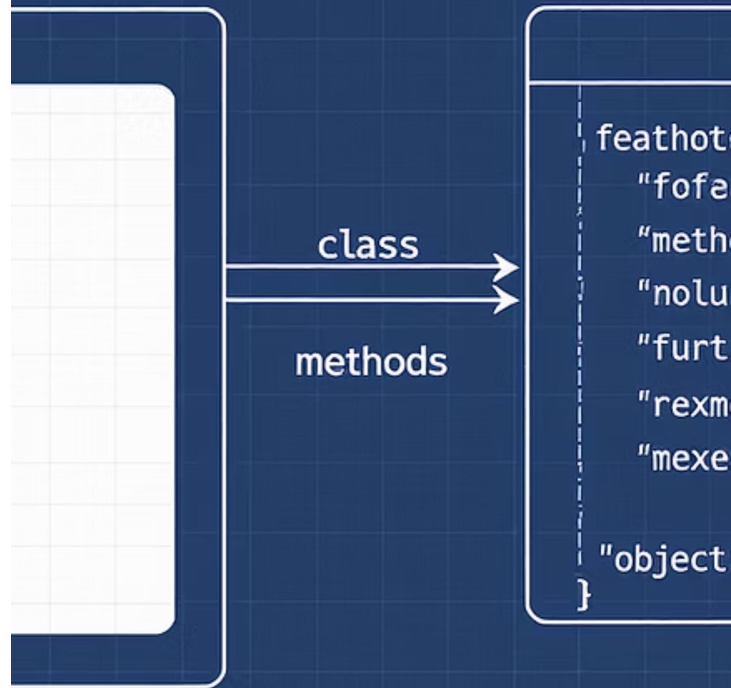


Comportamiento y Métodos

Los **métodos** son funciones definidas dentro de una clase que determinan el **comportamiento** de un objeto. Cada método representa una acción que un objeto puede realizar y suele operar sobre los atributos del objeto.

En Python, todos los métodos de una clase deben recibir el parámetro `self` como primer argumento, que representa la instancia actual del objeto y permite acceder a sus atributos y otros métodos.

Methods within 1 class (Python class)



11, class



Ejemplo de métodos

Por ejemplo, en la clase Perro, el método ladrar representa un comportamiento de los objetos Perro y puede ser llamado en cualquier instancia de la clase:

```
mi_perro = Perro("Rex", 3)
mi_perro.ladrar() # Imprime: ¡Guau!
```





Tipos de métodos en Python



Métodos públicos

Están disponibles para ser llamados desde fuera de la clase.



Métodos privados

Comienzan con un guion bajo y están destinados al uso interno de la clase. Los métodos privados ayudan a organizar el comportamiento interno del objeto sin exponer su implementación a otros objetos.



Métodos de clase

Se definen con el decorador `@classmethod` y reciben `cls` como primer argumento en lugar de `self`, lo que permite trabajar con la clase en lugar de una instancia específica.



Métodos estáticos

Definidos con `@staticmethod`, no reciben `self` ni `cls`, y se utilizan para realizar operaciones que no dependen del estado del objeto ni de la clase.



```
ttlections;
```

```
ot iclass={}
```

```
t((nstood) = metleclacy:));
```

```
[matlection" = "mustinq;;a omeq);
```

```
can("(hastlod") = "metwtod";
```

Importancia de los métodos

La definición de métodos permite encapsular comportamientos específicos dentro de una clase, lo que facilita la reutilización del código y hace que el programa sea más modular. Al igual que con los atributos, la separación de métodos públicos y privados ayuda a mantener el diseño de la clase limpio y ordenado.

En Python, el uso de métodos es fundamental para definir lo que un objeto puede hacer, y permite implementar comportamientos que respondan a la lógica del sistema, como cálculos, validaciones o interacciones con otros objetos.



Constructores e Instanciación

El **constructor** es un método especial que se llama automáticamente cuando se crea una nueva instancia de una clase. En Python, el constructor se define con el método `__init__`. Su propósito es inicializar los atributos del objeto cuando se crea, configurando el estado inicial de cada instancia de la clase.

El constructor recibe `self` y otros parámetros necesarios para asignar valores a los atributos.

```
tor =. cclass^;
```

```
trd(t.-f ment));  
t/cam..flori));
```

↑ *inin*
↓
clas met



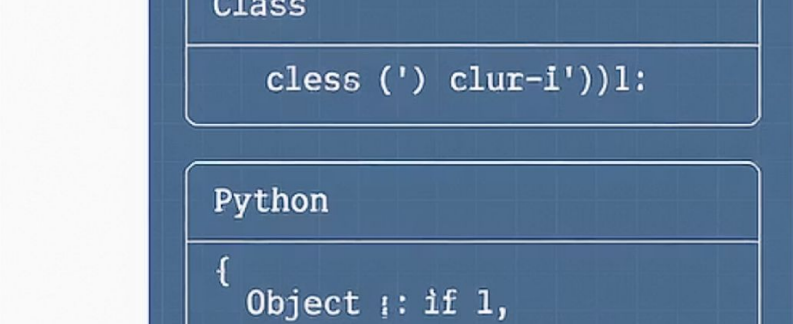
Ejemplo de constructor

Por ejemplo, en la clase Perro, el constructor permite asignar un nombre y una edad a cada nuevo perro que se cree:

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

mi_perro = Perro("Rex", 3)
```





Proceso de instanciación

Aquí, el constructor `__init__` asigna nombre y edad a los atributos del objeto `mi_perro`. Esto permite que cada instancia de la clase `Perro` tenga un estado inicial diferente. La **instanciación** es el proceso de crear un objeto a partir de una clase, y se realiza llamando a la clase como si fuera una función.





Importancia del constructor



Estado inicial

El constructor es fundamental en la OOP, ya que define cómo se crean los objetos y asegura que tengan un estado inicial válido.



Prevención de errores

Sin un constructor, los objetos podrían ser creados sin atributos esenciales, lo que generaría errores en tiempo de ejecución.



Flexibilidad

Python permite definir constructores personalizados que acepten diferentes parámetros, brindando flexibilidad en la creación de objetos.



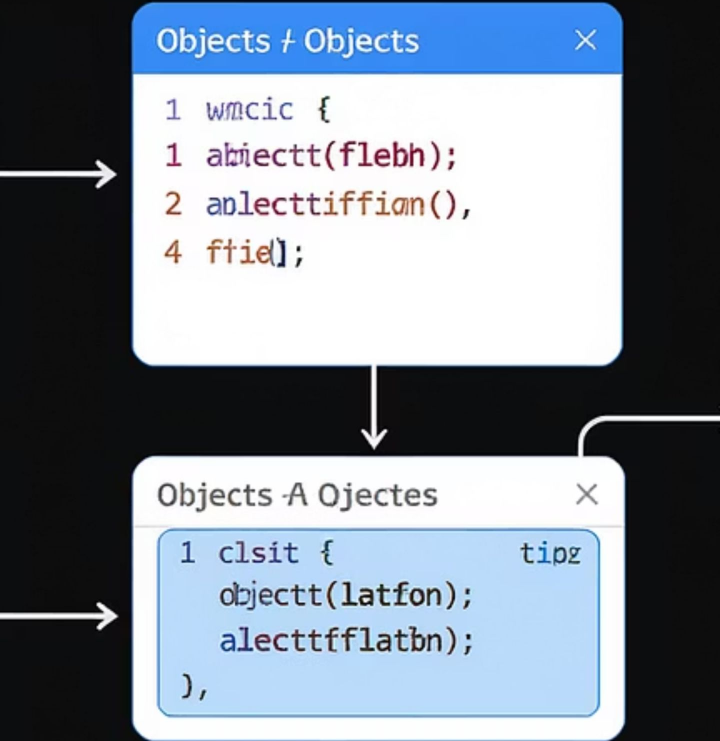


Métodos especiales adicionales

Además, se pueden definir métodos especiales adicionales en la clase, como `__str__` o `__repr__`, que controlan cómo se muestra el objeto cuando se imprime o se representa en consola. Esto permite personalizar la presentación de los objetos y mejorar la legibilidad del código.

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
    def __str__(self):
        return f"Perro: {self.nombre}, {self.edad} años"
```





Importancia de la instanciación

En Python, la instanciación y el uso de constructores son esenciales para trabajar con objetos en la OOP. Estos conceptos permiten que cada objeto tenga su propio estado y personalidad, y proporcionan un control preciso sobre cómo y cuándo se crean los objetos.

Live Coding

¿En qué consistirá la Demo?

Vamos a construir una clase `Empleado` que represente a una persona de una empresa. Se mostrará cómo definir atributos, crear métodos para mostrar información y actualizar datos, e instanciar múltiples objetos con sus propios estados.

1. Definir clase `Empleado` con `__init__()`
2. Atributos: `nombre`, `rol`, `salario`
3. Método `mostrar_info()` que imprima los datos
4. Método `actualizar_salario(nuevo_monto)` que modifique el atributo
5. Método especial `__str__()` para representación legible
6. Instanciar varios empleados y mostrar su información
7. Acceder a atributos y modificarlos desde fuera y dentro de métodos
8. Usar lista de objetos y recorrerla

Tiempo: 25 Minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Gestor de empleados con clases

Gestor de empleados con clases

Contexto: 🙌

Una empresa necesita un sistema simple que le permita gestionar a sus empleados. Cada empleado debe tener un nombre, un rol y un salario. Además, se debe poder actualizar su salario y verificar si gana por encima del promedio general.

Consigna: 📝

Codificar una rutina utilizando clases provistas para la resolución de un problema simple de acuerdo al paradigma de orientación a objetos en Python.

Tiempo 🕒: 30 Minutos

Gestor de empleados con clases

Paso a paso:

1. Definir una clase Empleado con atributos:
 - a. nombre (str), rol (str), salario (float)
2. Crear métodos:
 - mostrar() → imprime la información del empleado
 - actualizar_salario(nuevo_monto)
 - gana_mas_que(promedio) → devuelve True si gana más
3. Instanciar 3 empleados distintos y almacenarlos en una lista
4. Recorrer la lista:
 - Mostrar su información
 - Calcular el promedio de salario
 - Usar gana_mas_que() para informar quién está por encima del promedio

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprendimos los conceptos clave de la Programación Orientada a Objetos**
- ✓ **Definimos clases, atributos y métodos en Python**
- ✓ **Creamos e instanciamos objetos con su propio estado**
- ✓ **Aplicamos abstracción y encapsulación en el diseño de clases**
- ✓ **Modelamos un caso real con estructuras organizadas y reutilizables**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

